25AICP308 - Data Structures and Algorithms Lab

Course Objectives:

- To develop hands-on programming skills for implementing linear and non linear data structures.
- To apply and debug searching, sorting, and traversal algorithms in solving computational problems.
- To demonstrate the effective use of data structures for real-world problem solving through programming.

List of Exercises

1. Write a program to create a Stack using arrays and perform push, pop, peek, and display operations.
2. Write a program to implement a Singly Linked List and perform insert, delete, search, and reverse operations.
3. Write a program to implement a Circular Queue using arrays and perform enqueue and dequeue operations.
4. Write a program to implement Linear Search on an unsorted array and display the position of the searched element.
5. Write a program to implement Binary Search on a sorted array using both iterative and recursive methods.
6. Write a program to perform Breadth-First Search (BFS) and Depth-First Search (DFS) traversals on a graph.
7. Write a program to create a Binary Search Tree (BST) and perform insertion, deletion, and search operations.
8. Write a program to perform In-order, Pre-order, and Post-order traversals on a Binary Tree.
9. Write a program to implement Bubble Sort, Selection Sort, and Insertion Sort algorithms and compare their performance.
10. Write a program to implement Quick Sort, Merge Sort, and Heap Sort algorithms and compare their performance.

Course Outcomes:

At the end of this course, the students will be able to

1. Develop C++ programs for implementing fundamental data structures like stacks, queues, and linked lists.
2. Apply searching, sorting, and traversal algorithms through programming.
3. Demonstrate the ability to implement advanced data structures like Trees, Heaps, and Graphs.

Mapping of Course Outcomes with Programme Outcomes and Programme Specific Outcomes														
	PO1	PO2	PO3	PO4	PiO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	2	2	-	-	-	-	-	-	-	-	-	-	
CO2	1	2	2	2	-	-	-	-	-	-	-	2	-	-
CO3	1	2	2	-	-	-	-	-	2	-	2	2	-	-

Table of Content

Ex. No.	Date	Name of the Exercise	Page No.	Marks	Sign
1.		Linear Searching on an unsorted array			
2.		Binary Searching on a sorted array a) by Iteration b) by Recursion			
3.		Operations on Queue and Circular Queue			
4.		Operations on Stack			
5.		Operations on Singly Linked List			
6.		a) Insertion Sort b) Bubble Sorting c) Selection Sorting d) Comparisons on their Performance			
7.		a) Merge Sorting b) Quick Sorting c) Heap Sorting d) Comparisons on their Performance			
8.		Binary Tree Traversals			
9.		Creation and Operations on Binary Search Tree Traversal			
10.		Graph Traversals by Breadth First Search and Depth First Search			
			Total:		
			Average:		

Ex. No. : 01
Date :

Linear Searching on an unsorted array

Aim:

To write a C++ program to search a key element from the given list by Linear search technique.

Concept:

A linear search also known as a sequential search is the most straightforward algorithm used to find a target value within an unordered list or unsorted array.

It works by checking each element of the collection one by one from the very beginning until a match is found or the end of the collection is reached.

Procedure:

Member variables in the class Linear

n and key as integer

a as array of integer of size 100

Member functions in the class Linear

void GetData(void) - To get the input from user

void LinearSearch(void) - To search the key in the given array a using Linear search technique

Algorithm

void GetData(void)

1. Get the list of numbers.
2. Get the Key value to be searched.

void LinearSearch(void)

1. Compare the key value with the numbers in the list from the first position.
2. If the key value is present in the list, then print key value and its position.
3. If the key value is not present in any position of the list then, print “The key element is not present in the list”.

int main()

1. Create an object L of class Linear.
2. Call member function GetData() of object L.
3. Call member function LinearSearch() of object L.

Source Code:

```
// Linear Search
#include <iostream.h>
using namespace std;

class Linear
{
        int a[100],n,key;
  public:
        void GetData(void);
        void LinearSearch(void);
};

void Linear::GetData(void)
{
  cout<<"\nEnter the size of the list to be searched: ";
  cin>>n;
  cout<<"\nEnter the elements of the list: ";
  for(int i=0;i<n;i++)
 {
  cin>>a[i];
 }
 cout<<"Enter the key value to be searched in the above list: ";
 cin>>key;
}

void Linear::LinearSearch(void)
{
  for(int i=0;i<n;i++)
 {
   if(a[i]==key)
  {
    cout<<"The key value "<<key<<" is present at location "<<i+1;
    return;
  }
 }
 cout<<"The key value "<<key<<" is not present in the above list";
}

int main()
{
  Linear L;
  L.GetData();
  L.LinearSearch();
  return 0;
}
```

Sample Input/Output:

Enter the size of the list to be searched: 11
Enter the elements of the list: -12 23 1 -15 3 5 2 4 4 66 -11
Enter the key value to be searched in the above list: 3
The key value 3 is present at location 5

Result:

Thus, a C++ program to search a key element from the given list by linear search technique has been successfully written and outputs have been verified.

Ex. No. : 02
Date :

Binary Searching on a Sorted Array

Ex. No. 2(a) **by Iteration**

Aim:

To write a C++ program to search a key element from the given list by binary search technique.

Concept:

Binary Search is an efficient searching algorithm designed to locate a specific target value within a sorted data structure by repeatedly halving the search space.

Unlike a linear search which checks elements sequentially, binary search uses a divide-and-conquer strategy. This brings its worst-case time complexity down to a highly efficient O(log n).

Conditions to apply Binary Search Algorithm in a Data Structure

1. The data structure must be sorted.
2. Access to any element of the data structure should take constant time.

Binary Search Algorithm

1. Divide the search space into two halves by **finding the middle index "mid"**.
2. Compare the middle of the search space with the **key**.
3. If the **key** is found at middle, the process is terminated.
4. If the **key** is not found at middle, choose which half will be used as the next search space.
   -> If the **key** is smaller than the middle, then the **left** side is used for next search.
   -> If the **key** is larger than the middle, then the **right** side is used for next search.
5. This process is continued until the **key** is found or the total search space is exhausted.

Procedure:

Member variables in the class Binary

n and key as integer
a as array of integer of size 100

Member functions in the class Binary

void GetData(void)	-	To get the input from user
void Sort(void)	-	To sort the input in ascending order
void Display(void)	-	To display all elements of the list
void BinarySearch(void)	-	To search the key in the given array a using binary search technique

Algorithm

void GetData(void)

1. Get the list of numbers.
2. Get the Key value to be searched.

void Sort(void)

1. for i=0 to n-1 do step 2
2. for j=i+1 to n do step 3
3. if(a[i]>a[j]) exchange a[i] and a[j]

void Display(void)

1. Display elements of the list one by one.

void BinarySearch(void)

1. Assign:
    a. high= total number of elements in the list -1.
    b. low=0.
2. until low <= high repeat the following steps,
    a. mid= (high + low)/2.
    b. Check the key value with the value present at the middle index position of the sorted list.
    c. If present, display its position in the list and stop the process.
    d. Else if key value < the value present in the middle position, then
        i. high=mid-1;
    e. Else if key value > the value present in the middle position, then
        i. low=mid+1;
3. Display “The key value is not present in the list”.

int main()

1. Create an object B of class Binary.
2. Call member function GetData() of object B.
3. Call member function Sort() of object B.
4. Call member function Display() of object B.
5. Call member function BinarySearch() of object B.

Source Code:

```
// Binary Search - Iteration
#include <iostream>
using namespace std;

class Binary
{
  private:
        int a[100], n, key;
  public:
        void GetData(void);
        void Sort(void);
        void Display(void);
        void BinarySearch(void);
};

void Binary::GetData(void)
{
  cout<<"\nEnter the size of the list to be searched: ";
  cin>>n;
  cout<<"\nEnter the elements of the list:";
  for(int i=0;i<n;i++)
  {
    cin>>a[i];
  }
  cout<<"Enter the key value to be searched in the above list: ";
  cin>>key;
}

void Binary::Sort(void)
{
  for(int i=0;i<n-1;i++)
  for(int j=i+1;j<n;j++)
  {
   if(a[i]>a[j])
  {
    int temp=a[i];
    a[i]=a[j];
    a[j]=temp;
   }
  }
}

void Binary::Display(void)
{
```

```
  for(int i=0;i<n;i++)
  cout<<" Element "<< i+1 <<" is " << a[i]<<endl;
}

void Binary::BinarySearch(void)
{
  int high, low, mid;
  high=n-1;
  low=0;
  while (low<=high)
  {
    mid=(low + high)/2;
    if(a[mid]==key)
   {
     cout<<"\n The key value "<< key <<" is present at location " <<mid+1;
     return;
    }
    else if (key<a[mid])
   {
     high=mid-1;
    }
    else
   {
     low=mid+1;
    }
  }
  cout<<"\n The key value "<< key<<" is not present in the above list";
}

int main()
{
  Binary B;
  B.GetData();
  B.Sort();
  cout<<"\n The Sorted List \n";
  cout<<" ----------------- \n";
  B.Display();
  B.BinarySearch();
  return 0;
}
```

Sample Input / Output:

Enter the size of the list to be searched: 11
Enter the elements of the list: -12 23 1 -15 3 5 2 4 4 66 -11
Enter the key value to be searched in the above list: 3

```
The Sorted List
-----------------
Element 1 is -15.
Element 2 is -12.
Element 3 is -11.
Element 4 is 1.
Element 5 is 2.
Element 6 is 3.
Element 7 is 4.
Element 8 is 4.
Element 9 is 5.
Element 10 is 23.
Element 11 is 66.
The key value 3 is present at location 6.
```

Result:

Thus, a C++ program to search a key element from the given list by binary search technique using iteration has been successfully written and outputs have been verified.

Ex. No. : 02
Date :

Binary Searching on a Sorted Array

Ex. No. 2(b)

by Recursion

Aim:

To write a C++ program to search a key element from the given list by binary search technique.

Concept:

Binary Search is an efficient searching algorithm designed to locate a specific target value within a sorted data structure by repeatedly halving the search space.

Unlike a linear search which checks elements sequentially, binary search uses a divide-and-conquer strategy. This brings its worst-case time complexity down to a highly efficient O(log n).

Conditions to apply Binary Search Algorithm in a Data Structure

1. The data structure must be sorted.
2. Access to any element of the data structure should take constant time.

Binary Search Algorithm

1. Divide the search space into two halves by **finding the middle index "mid"**.
2. Compare the middle of the search space with the **key**.
3. If the **key** is found at middle, the process is terminated.
4. If the **key** is not found at middle, choose which half will be used as the next search space.
   -> If the **key** is smaller than the middle, then the **left** side is used for next search.
   -> If the **key** is larger than the middle, then the **right** side is used for next search.
5. This process is continued until the **key** is found or the total search space is exhausted.

Procedure:

Member variables in the class Binary

n and key as integer
a as array of integer of size 100

Member functions in the class Binary

void GetData(void)	-	To get the input from user
void Sort(void)	-	To sort the input in ascending order
void Display(void)	-	To display all elements of the list
void BinarySearch(int, int)	-	To search the key in the given array a using binary search technique using recursive function

Algorithm

void GetData(void)

1. Get the list of numbers.
2. Get the Key value to be searched.

void Sort(void)

1. for i=0 to n-1 do step 2
2. for j=i+1 to n do step 3
3. if(a[i]>a[j]) exchange a[i] and a[j]

void Display(void)

1. Display elements of the list one by one.

void BinarySearch(int low, int high)

1. If low > high goto step 3.
2. until low <= high repeat the following steps,
    a. mid= (high + low)/2.
    b. Check the key value with the value present at the middle index position of the sorted list.
    c. If present, display its position in the list and stop the process.
    d. Else if key value < the value present in the middle position, then
        i. Call the function BinarySearch(low, mid-1).
    e. Else if key value > the value present in the middle position, then
        i. Call the function BinarySearch(mid+1, high).
3. Display “The key value is not present in the list”.

int main()

1. Create an object B of class Binary.
2. Call member function GetData() of object B.
3. Call member function Sort() of object B.
4. Call member function Display() of object B.
5. Call member function BinarySearch(mid+1, high) of object B.

Source Code:

```
// Binary Search - Recursion
#include <iostream>
using namespace std;

class Binary
{
  private:
        int a[100],n,key;
  public:
        void GetData(void);
        void Sort(void);
        void Display(void);
        void BinarySearch(void);
};

void Binary::GetData(void)
{
  cout<<"\nEnter the size of the list to be searched: ";
  cin>>n;
  cout<<"\nEnter the elements of the list:";
  for(int i=0;i<n;i++)
  {
    cin>>a[i];
  }
  cout<<"Enter the key value to be searched in the above list: ";
  cin>>key;
}

void Binary::Sort(void)
{
  for(int i=0;i<n-1;i++)
  for(int j=i+1;j<n;j++)
  {
   if(a[i]>a[j])
  {
    int temp=a[i];
    a[i]=a[j];
    a[j]=temp;
   }
  }
}

void Binary::Display(void)
{
```

```
  for(int i=0;i<n;i++)
  cout<<" Element "<< i+1 <<" is " << a[i]<<endl;
}

void Binary::BinarySearch(void)
{
  int high,low,mid;
  high=n-1;
  low=0;
  while (low<=high)
  {
    mid=(low+high)/2;
    if(a[mid]==key)
   {
     cout<<"\n The key value "<< key <<" is present at location " <<mid+1;
     return;
    }
    else if (key<a[mid])
   {
     high=mid-1;
    }
    else
   {
     low=mid+1;
    }
  }
  cout<<"\n The key value "<< key<<" is not present in the above list";
}

int main()
{
  Binary B;
  B.GetData();
  B.Sort();
  cout<<"\n The Sorted List \n";
  cout<<" ----------------- \n";
  B.Display();
  B.BinarySearch();
  return 0;
}
```

Sample Input/Output:

Enter the size of the list to be searched: 11

Enter the elements of the list: 4 7 23 98 23 56 3 59 1 87 6

Enter the key value to be searched in the above list: 3

```
The Sorted List
----------------
Element 1 is 1
Element 2 is 3
Element 3 is 4
Element 4 is 6
Element 5 is 7
Element 6 is 23
Element 7 is 23
Element 8 is 56
Element 9 is 59
Element 10 is 87
Element 11 is 98

The key value 3 is present at location 2
```

Result:

Thus, a C++ program to search a key element from the given list by binary search technique using recursion has been successfully written and outputs have been verified.

Ex. No. : 03(a)
Date :

Queue Operations

Aim:

To write a C++ program to implement a queue data structure.

Concept:

A Queue is a linear data structure that operates on the First In, First Out (FIFO) principle, meaning the first element added to the queue will be the first one to be removed. It mimics a real-world waiting line, such as people waiting in line at a ticket counter. Unlike a Stack (which uses Last In, First Out), a queue is open at both ends.

Concept:

Enqueue:

Adds an item to the Rear of the queue. If the queue is full, it causes an Overflow condition.

Dequeue:

Removes and returns the item from the Front of the queue. If the queue is empty, it causes an Underflow condition.

Peek / Front:

Returns the element at the front without actually removing it.

isEmpty:

Checks if the queue contains any elements.

isFull:

Checks if the queue has reached its maximum allocated capacity.

Procedure:

Member variables in the class Queue

size, front, rear as integer
QueueData as array of integer of size 100

Member functions in the class Queue

Queue( )	- Constructor to set front and rear as 0
void SetSize(int n)	- To set the size of queue
void Enque(int);	- To insert value x into queue
int Deque(void);	- To remove a value from queue
void View(void);	- To display all elements of queue

Algorithm

Queue()

1. Initialize the front and rear to 0.

void SetSize(int n)

1. Set the size of the Queue as n.

void Enque(int x)
1. Check whether there is any space for new element to be added.
2. If there is any space then, place the new element in the position pointed by rear and increment the rear by one.
3. Otherwise, display “Queue is over flow”.

int Deque(void)
1. Check whether there is any element present in the queue.
2. If there is an element then, store the element present at the location pointed by front in x and increment the front by one and return the value of x.
3. Otherwise, display “Queue is under flow”.

void Display(void)
1. If there is element Display the elements present in the locations pointed by front to rear.
2. Otherwise, display “Queue is empty”.

int main()
1. Display menu to perform queue operations.
2. Get user choice and perform corresponding operation.

```
//Queue Operation
#include<iostream>
using namespace std;
class Queue
{
        int QueueData[100],size,front,rear;
public:
        Queue() {front=0;rear=0;}
        void SetSize(int n) {size=n;}
        void Enque(int);
        int Deque(void);
        void View(void);
};

void Queue::Enque(int x)
{
        if((rear-front+1)>size)
        {
                cout<<"\n\t\tQueue is over flow...";
        }
        else
        {
                QueueData[rear]=x;
                cout<<"\n\t\t"<<x<<" is added.\n";
                rear++;
```

```
        }
}

int Queue::Deque(void)
{
        int x;
        if(front>=rear)
        {
                cout<<"\n\t\tQueue is under flow...";
                return -9999;
        }
        else
        {
                x=QueueData[front];
                front++;
                return x;
        }
}

void Queue::View()
{
        int i=front,j=rear;
        if(i>=j)
        {
                cout<<"\n\t\tQueue is empty...";
        }
        else
        {
                cout<<"\n\n\tData in Queue :";
                for(;i<=j-1;i++)
                        cout<<" "<<QueueData[i];
        }
}
int main()
{
        int n;
        cout<<"\nEnter the size of Queue [Max=100] : ";
        cin>>n;
        Queue Q;
        Q.SetSize(n);
        int choice;
        do
        {
                cout<<"\n\t\tQueue Operation";
                cout<<"\n\t\t~~~~~~~~~~~~~~~";
                cout<<"\n\n\t1.Insert\n\t2.Delete\n\t3.View\n\t4.Exit";
```

```
                cout<<"\n\nEnter your choice : ";
                cin>>choice;
                int x;
                switch(choice)
                {
                        case 1:
                                cout<<"\nEnter a value : ";
                                cin>>x;
                                Q.Enque(x);
                                Q.View();
                                break;

                        case 2: x=Q.Deque();
                                if (x!=-9999)
                                        cout<<"\n\n\t"<<x<<" is deleted.\n\n";
                                Q.View();
                                break;

                        case 3:
                                Q.View();
                                break;

                        case 4:
                                cout<<"\n\n\t\t Exit...";
                                break;
                }
        }while(choice!=4);
        return 0;
}
```

Sample Input/Output:

Enter the size of Queue [Max=100]: 3

Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 1

Enter a value : 11

11 is added.
~~~~~~~~~~~~~

Data in Queue: 11

Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 1

Enter a value : 22

22 is added.

Data in Queue: 11 22

Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 1

Enter a value : 33

33 is added.

Data in Queue: 11 22 33

Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 1

Queue is over flow...

Data in Queue: 11 22 33
~~~~~~~~~~~~~

Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 3

Data in Queue : 11 22 33

Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 2

11 is deleted.

Data in Queue : 22 33

Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 2

22 is deleted.

Data in Queue: 33

Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 2

33 is deleted.

Queue is empty...

Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 2

Queue is under flow...

Queue is empty…

Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 4

Exit...

Result:

Thus, a C++ program to implement the queue data structure has been successfully written and its operations have been verified.

Ex. No. : 03(b)
Date :

Circular Queue Operations

Aim:

To write a C++ program to implement a circular queue data structure.

Concept:

A circular queue is an advanced linear data structure that operates on the First In First Out (FIFO) principle, where the last position is logically connected back to the first position to form a circle. This design—often called a Ring Buffer—solves the major limitation of a standard linear queue, where empty spaces created by deletions at the front cannot be reused once the rear pointer reaches the end of the array.

Instead of linearly incrementing pointers, a circular queue relies on **modular arithmetic** to wrap pointers back to index `0` when they exceed `capacity - 1`

Pointers:

It maintains two integer markers: `front` (tracks the removal point) and `rear` (tracks the insertion point). Both start initialized to `-1`.

Enqueue (Insertion):

Adds an item to the back of the queue. The index is calculated as:

rear = (rear + 1) (modulus of size of the circular queue)

Dequeue (Deletion):

Removes an item from the front. The index is advanced using:

front = (front + 1) (modulus of size of the circular queue)

Empty Queue:

The queue is empty when front = = -1.

Full Queue:

The queue is full when the next circular slot of rear hits front. Mathematically.

(rear +1) (modulus of size of the circular queue) = = front

Procedure:

Member variables in the class Queue

size, front, rear as integer
QueueData as array of integer of size 100

Member functions in the class Queue

Queue( )	- Constructor to set front and rear as 0
void SetSize(int n)	- To set the size of queue
void Enque(int);	- To insert value x into queue
int Deque(void);	- To remove a value from queue

void View(void); - To display all elements of queue

Algorithm

Queue()
1. Initialize the front and rear to 0.

void SetSize(int n)
1. Set the size of the Queue as n.

void Enque(int x)
1. Check whether there is any space for new element to be added.
2. If there is any empty space then, place the new element in the position pointed by rear and increment the rear by one. **[i.e. (rear + 1) % size of the queue n]**
3. Otherwise, display "Queue is over flow".

int Deque(void)
1. Check whether there is any element present in the queue.
2. If there is an element then, store the element present at the location pointed by front in x and increment the front by one **[i.e. (front + 1) % size of the queue n]** and return the value of x.
3. Otherwise, display "Queue is under flow".

void Display(void)
1. If there is element Display the elements present in the locations pointed by front to rear.
2. Otherwise, display "Queue is empty".

int main()
1. Display menu to perform queue operations.
2. Get user choice and perform corresponding operation.

Source Code:

```
// Circular Queue Operation
#include<iostream>
using namespace std;

class Queue
{
private:
        int QueueData[100], size, front, rear, count;
public:
        Queue()
        {
          front = 0;
          rear = 0;
          count = 0; // Tracks the current number of elements
        }
        void SetSize(int n)
```

```
        {
          size = n;
        }
        void Enque(int);
        int Deque(void);
        void View(void);
};

void Queue::Enque(int x)
{
 // Check if the queue is full using the element counter
 if(count == size)
{
  cout<<"\n\t\tQueue is overflow...";
 }
 else
{
  QueueData[rear] = x;
  cout<<"\n\t\t"<<x<<" is added.\n";
  rear = (rear + 1) % size; // Wrap around to the beginning if needed
  count++; // Increase element count
 }
}

int Queue::Deque(void)
{
  int x;
// Check if the queue is empty using the element counter
  if(count == 0)
 {
   cout<<"\n\t\tQueue is underflow...";
   return -9999;
  }
  else
 {
   x = QueueData[front];
   front = (front + 1) % size; // Wrap around to the beginning if needed
   count--; // Decrease element count
   return x;
  }
}

void Queue::View()
{
  if(count == 0)
 {
```

```
   cout<<"\n\t\tQueue is empty...";
  }
  else
 {
   cout<<"\n\n\tData in Queue :";
   int i = front;
   for(int c = 0; c < count; c++)
   {
     cout<<" "<<QueueData[i];
      i = (i + 1) % size; // Traverse circularly
   }
  }
}

int main()
{
  int n;
  cout<<"\nEnter the size of Queue [Max=100] : ";
  cin>>n;

// Prevent array out-of-bounds if user enters more than 100
   if (n > 100)
    n = 100;

    Queue Q;
    Q.SetSize(n);
    int choice;

    do {
        cout<<"\n\t\tCircular Queue Operation";
        cout<<"\n\t\t~~~~~~~~~~~~~~~~~~~~~~~~~~";
        cout<<"\n\n\t1.Insert\n\t2.Delete\n\t3.View\n\t4.Exit";
        cout<<"\n\nEnter your choice : ";
        cin>>choice;
        int x;

        switch(choice)
        {
            case 1:
                cout<<"\nEnter a value : ";
                cin>>x;
                Q.Enque(x);
                Q.View();
                break;
            case 2:
                x=Q.Deque();
```

```
            if (x!=-9999) cout<<"\n\n\t"<<x<<" is deleted.\n\n";
            Q.View();
            break;
        case 3:
            Q.View();
            break;
        case 4:
            cout<<"\n\n\t\t Exit...";
            break;
     }
   }while(choice!=4);
    return 0;
}
```

Sample Input/Output:

Enter the size of Queue [Max=100]: 3

Circular Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 1

Enter a value : 11

11 is added.

Data in Queue: 11

Circular Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 1

Enter a value : 22

22 is added.

Data in Queue: 11 22

Circular Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 1

Enter a value : 33

33 is added.

Data in Queue: 11 22 33

Circular Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 1

Queue is over flow...

Data in Queue: 11 22 33

Circular Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 3

Data in Queue : 11 22 33

Circular Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View

4.Exit

Enter your choice: 2

11 is deleted.

Data in Queue : 22 33

Circular Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 2

22 is deleted.

Data in Queue: 33

Circular Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 2

33 is deleted.

Circular Queue is empty...

Circular Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 2

Circular Queue is under flow...

Circular Queue is empty…
~~~~~~~~~~~~~

Circular Queue Operation
~~~~~~~~~~~~~
1.Insert
2.Delete
3.View
4.Exit

Enter your choice: 4

Exit...

Result:

Thus, a C++ program to implement the circular queue data structure has been successfully written and its operations have been verified.
~~~~~~~~~~~~~

Ex. No. : 04
Date :

Stack Operations

Aim:

To write a C++ program to implement a Stack data structure.

Concept:

A stack is a linear data structure that follows the Last In, First Out (LIFO) principle, meaning the element added most recently is always the first one to be removed. Think of it like a physical stack of plates: you can only add a new plate to the top, and you can only take a plate off from the top.

All data operations occur at a single end called the Top of the Stack (TOS). The primary operations include:

Push	: Inserts an element onto the top of the stack.
Pop	: Removes and returns the topmost element.
Peek / Top	: Returns the topmost element without removing it.
IsEmpty	: Checks if the stack has no elements.
IsFull	: Checks if a fixed-size stack has reached its maximum capacity.

Procedure:

Member variables in the class Stack

size, top as integer
StackData as array of integer of size 100

Member functions in the class Stack

Stack( ) {top=-1;}	- Constructor to set top as -1
void SetSize(int n)	- To set the size of stack
void Push(int x)	- To insert value x into stack
int Pop(void)	- To remove a value from stack
void Display(void)	- To display all elements of stack

Algorithm:

Stack()

1. Initialize the top to –1.

void SetSize(int n)

1. Set the size of the Stack as n

void Push(int x)

1. Check whether the top is less than the size of the stack.
2. If there is space then, increment the top by one and place the new element in the position pointed by top.
3. Otherwise, display “Stack is over flow”.

int Pop(void)
1. Check whether the top is greater than –1.
2. If there is an element then, store the element present at the location pointed by top in x and decrement the value of the pointer by one and return the value of x.
3. Otherwise, display “Stack is under flow”.

void Display(void)
1. Display the elements present in the locations pointed by top to location zero.
2. Otherwise, display “Stack is empty”.

int main( )
1. Display menu to perform Stack operations.
2. Get user choice and perform corresponding operation.

//Source Code:

```
//Stack Operations
#include<iostream>
using namespace std;
class Stack
{
 private:
          int StackData[100],size,top;
 public:
  Stack() {top=-1;}
  void SetSize(int n) {size=n;}
  void Push(int x);
  int Pop(void);
  void Display(void);
};

void Stack::Push(int x)
{
 if(top>=size-1)
 {
  cout<<"\n\t\tStack is over flow...";
 }
 else
 {
  top++;
  StackData[top]=x;
  cout<<"\n\t\t"<<x<<" is pushed \n\n\n";
 }
}

int Stack::Pop(void)
{
```

```
 int x;
 if(top<=-1)
 {
  cout<<"\n\t\tStack is under flow";
  return -9999;
 }
 else
 {
  x=StackData[top];
  top--;
  return x;
 }
}

void Stack::Display(void)
{
 if(top>=0)
 {
  cout<<"\nThe elements in stack.";
  cout<<"\n~~~~~~~~~~~~~~~~~~~~~~~~";
  for(int i=top;i>=0;i--)
   cout<<"\n\nData "<<i+1<<" = "<<StackData[i];
   cout<<"\n\n\n\n Press any key to continue...";
 }
 else
 {
  cout<<"\nThe Stack is empty.";
 }
}

int main()
{
 Stack S;
 int option,n;
 cout<<"\nEnter the size of stack [Max=100] : ";
 cin>>n;
 S.SetSize(n);
 do
 {
  cout<<"\n\t\tStack Operation";
  cout<<"\n\t\t~~~~~~~~~~~~~~~~";
  cout<<"\n\t1.Push\n\t2.Pop\n\t3.Display\n\t4.Exit";
  cout<<"\n\nEnter your option : ";
  cin>>option;
  int x,topvalue;
  switch(option)
```

```
 {
  case 1:
        cout<<"\nEnter a value to be pushed : ";
        cin>>x;
        S.Push(x);
        S.Display();
        break;
  case 2:
         topvalue=S.Pop();
         if (topvalue!=-9999)
                cout<<"\n\t\t The popped element is "<<topvalue;
        S.Display();
        break;
  case 3:
        S.Display();
        break;
  case 4:
        cout<<"\n\n\t\t You are on EXIT...";
        break;
  default:
        cout<<"\n\n\t Enter only 1,2,3,4.";
}
}while(option!=4);
return 0;
}
```

Sample Input/Output:

```
Enter the size of stack [Max=10] : 5
Stack Operation
~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit

Enter your option : 1

Enter a value to be pushed : 5

5 is Pushed

The elements in stack.
~~~~~~~~~~~~~~~
Data 1 = 5
Press any key to continue...
```

```
Stack Operation
~~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit

Enter your option : 1

Enter a value to be pushed : 4

4 is Pushed

The elements in stack.
~~~~~~~~~~~~~~~~
Data 2 = 4
Data 1 = 5

Press any key to continue...

Stack Operation
~~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit
Enter your option : 1

Enter a value to be pushed : 3

3 is Pushed

The elements in stack.
~~~~~~~~~~~~~~~~
Data 3 = 3
Data 2 = 4
Data 1 = 5

Press any key to continue...

Stack Operation
~~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit
```

Enter your option : 1

Enter a value to be pushed : 22

22 is Pushed

The elements in stack.
~~~~~~~~~~~~~~~~
Data 4 = 22
Data 3 = 3
Data 2 = 4
Data 1 = 5

Press any key to continue...

Stack Operation
~~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit

Enter your option : 1

Enter a value to be pushed : 11

11 is Pushed

The elements in stack.
~~~~~~~~~~~~~~~~
Data 5 = 11
Data 4 = 22
Data 3 = 3
Data 2 = 4
Data 1 = 5

Press any key to continue...

Stack Operation
~~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit

Enter your option : 1

```
Stack is over flow...

Press any key to continue...

Stack Operation
~~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit

Enter your option : 2

The popped element is 11.

The elements in stack.
~~~~~~~~~~~~~~~~
Data 4 = 22
Data 3 = 3
Data 2 = 4
Data 1 = 5

Press any key to continue...

Stack Operation
~~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit

Enter your option : 2

The popped element is 22.

The elements in stack.
~~~~~~~~~~~~~~~~
Data 3 = 3
Data 2 = 4
Data 1 = 5

Press any key to continue...

Stack Operation
~~~~~~~~~~~~
```

1.Push
2.Pop
3.Display
4.Exit

Enter your option : 2

The popped element is 3.

The elements in stack.
~~~~~~~~~~~~~~~~
Data 2 = 4
Data 1 = 5

Press any key to continue...

Stack Operation
~~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit

Enter your option : 2

The popped element is 4.

The elements in stack.
~~~~~~~~~~~~~~~~
Data 1 = 5

Press any key to continue...

Stack Operation
~~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit

Enter your option : 2

The popped element is 5.

The Stack is empty.

```
Stack Operation
~~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit

Enter your option : 2

Stack is under flow

The Stack is empty.

Stack Operation
~~~~~~~~~~~~
1.Push
2.Pop
3.Display
4.Exit

Enter your option : 4

You are on Exit...
```

Result:

Thus, a C++ program to implement the Stack data structure has been successfully written and its operations have been verified.

Basic C++ concepts

1. What is a variable? How are they declared, defined and used in programming language?''
2. What is an identifier?
3. List down the rules to be followed for naming an identifier in a programming language.
4. What is a constant?
5. What is a data type?
6. List down the various data types and its ranges supported by C++ language.
7. What is meant by arithmetic expression and algebraic expression?
8. What is meant by assignment statement?
9. What is the syntax of if…else statement? Explain with an example.
10. What is the syntax of for looping structure? Explain with an example.
11. What is the syntax of while looping structure? Explain with an example.
12. What is iteration?
13. What is the syntax of switch…case statement? Explain with an example.
14. Differentiate between function declaration and definition.
15. What is argument with respect to function?
16. Differentiate between actual and formal argument.
17. What is recursion?
18. What are arrays? How are they declared and accessed?
19. What is Class? How are they created?
20. What is Object? How are they created?
21. Differentiate between class declaration and class definition.
22. What is a member function? How are they accessed from outside the class?
23. What is the use of scope resolution operator?
24. What is encapsulation? What are the advantages of using it in the OOP language?
25. Explain the importance of using access modifier public and private in class declaration.

Linear Searching and binary searching

1. What is a list?
2. Differentiate between ordered and unordered list?
3. What is sorting?
4. What is swapping?
5. Define searching.
6. Why sorting is not necessary for linear searching?
7. Why binary searching needs a sorted list before searching?
8. What is iteration? Explain with an example.
9. Define array. Explain with an example.
10. What is recursion? Explain how it is implemented with example.
11. What is the time complexity and space complexity of linear searching?
12. What is the time complexity and space complexity of binary searching?
13. Compare best case and worst case analysis of searching an element with linear searching and binary searching methods.
14. Define data.
15. Define information
16. Define data structure.

17. What is linear data structure?
18. What is non-linear data structure?
19. List down the linear data structure.
20. List down the non-linear data structure.
21. Define queue.
22. List out the operations of queue data structure
23. What is enque?
24. What is deque?
25. What is the strategy followed in queue?
26. What is circular queue?
27. What is the strategy followed in circular queue?
28. At what end the elements are added and removed in queue? Name its operations.
29. What is a mathematical function used in circular queue?
30. Name the operations performed at front and rear end of the queue.
31. Name the real time usage of a queue in day today life.
32. What is stack?
33. List down the operations of stack.
34. What is the strategy followed in stack?
35. What is over flow? When will this happen?
36. What is underflow? When will this happen?
37. Where the stack data structure is used in programming?
38. Name the real time usage of a stack in day today life.
39. How many ends are active in stack data structure?
40. At what end the elements are added and removed in a stack?
41. What data structure is used in recursive function call?
42.